Project - High Level Design

on

Entertainment Planning Assistant Agent

Course Name: AGENTIC AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	MOHIT KUMAR GUPTA	EN22CS301607
2	MOHIT CHAUHAN	EN22CS301605
3	MOULI SUKHLA	EN22CS301615
4	NIKHIL JAVERY	EN22CS301647
5	NAMAN PANDEY	EN22CS301630

Group Name: Group 05D6

Project Number: AAI-14

Industry Mentor Name:

University Mentor Name: NISHANT SHRIVASTAV

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects
8. References

1. Introduction

The **MultiAgent Automotive System** is an intelligent, distributed multi-agent architecture designed to automate automotive research and comparative analysis using Artificial Intelligence. The system leverages specialized AI agents that collaborate in a coordinated manner to transform unstructured natural language queries into comprehensive, structured automotive comparison reports.

Traditional automotive comparison platforms rely on manually curated databases or static content, which often lack adaptability, real-time updates, and contextual intelligence. In contrast, the proposed system introduces an autonomous agent-based framework capable of:

- Interpreting user intent from natural language input
- Dynamically gathering data from multiple trusted sources
- Validating and structuring heterogeneous automotive data
- Generating professional comparison reports in multiple output formats

The system aims to bridge the gap between raw automotive data and actionable insights by automating the complete research-to-report pipeline.

By integrating modern AI orchestration techniques inspired by distributed intelligent systems research, the MultiAgent Automotive platform demonstrates how specialized agents can collaborate efficiently to solve complex information synthesis tasks.

1.1 Scope of the Document

This High-Level Design (HLD) document provides a comprehensive architectural overview of the MultiAgent Automotive system. It defines:

- System architecture and component hierarchy
- Functional and non-functional requirements
- Data flow and interaction models
- Agent responsibilities and orchestration strategy
- API design and external integrations
- Security, scalability, and performance considerations

The document serves as a strategic blueprint for transitioning from conceptual design to implementation. It outlines key architectural decisions, underlying assumptions, constraints, and trade-offs that influence system behavior.

This document does not include low-level implementation details such as class diagrams, internal method logic, or specific deployment scripts. Those aspects are addressed in the Low-Level Design (LLD) documentation.

1.2 Intended Audience

This document is intended for multiple stakeholder groups with varying technical backgrounds:

Development Team

To understand architectural boundaries, component responsibilities, communication mechanisms, and integration points required for implementation.

Project Managers

To estimate development timelines, identify resource requirements, evaluate technical risks, and monitor project milestones.

Quality Assurance Teams

To design structured test cases based on defined workflows, validate system reliability, and verify performance targets.

University Mentors

To assess architectural soundness, modularity, scalability considerations, and adherence to software engineering best practices.

Industry Mentors

To evaluate real-world applicability, system robustness, integration feasibility, and production-readiness.

Stakeholders and Evaluators

To gain a conceptual understanding of the system's purpose, operational flow, and technological value proposition.

1.3 System Overview

The MultiAgent Automotive System operates as a coordinated multi-agent ecosystem consisting of three primary intelligent agents:

Supervisor Agent

The central orchestrator responsible for:

- Parsing and analyzing user queries
- Identifying intent and comparison parameters
- Coordinating workflow between agents
- Monitoring task execution
- Performing quality control before final delivery

The Supervisor Agent ensures logical sequencing and maintains system coherence.

Researcher Agent

Responsible for external data acquisition and validation. It performs:

- Web scraping of automotive sources
- API integration with structured vehicle databases
- Data normalization and cleaning
- Cross-source validation to ensure accuracy

The Researcher Agent transforms raw, heterogeneous data into structured information ready for synthesis.

Writer Agent

Responsible for transforming validated structured data into user-consumable reports. It:

- Generates structured Markdown documents
- Produces formatted PDF reports
- Applies comparison templates
- Ensures clarity, readability, and professional formatting

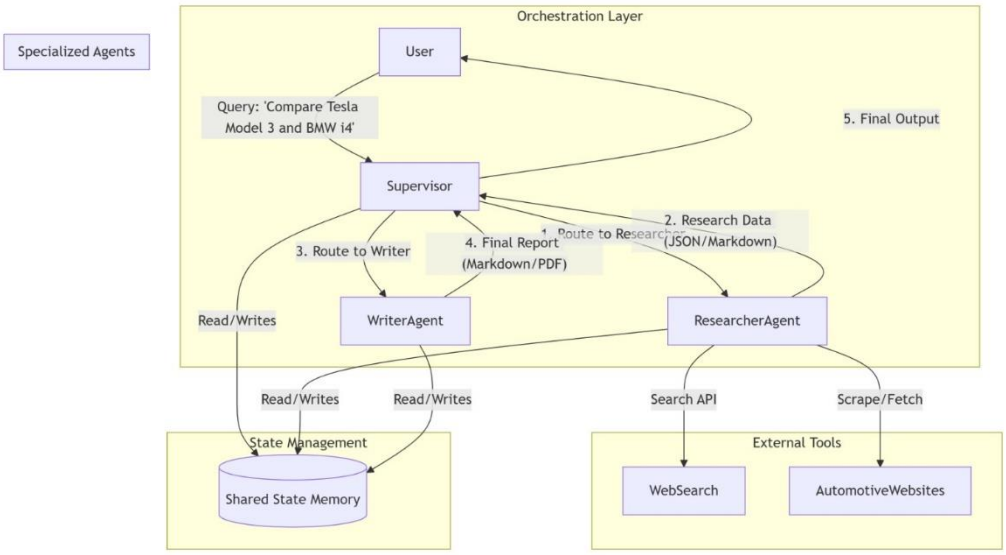
The Writer Agent bridges analytical results with presentation quality.

Shared State Memory System

All agents communicate through a centralized shared state memory layer that provides:

- Session management
- Data persistence
- Caching for performance optimization
- Fault recovery support

This decoupled communication model enhances scalability and fault tolerance by reducing direct inter-agent dependencies.



High-Level Architecture Overview

2. System Design

The Multi-Agent Automotive Information System follows a modular, service-oriented, multi-agent architecture. The system is designed to ensure scalability, flexibility, maintainability, and high performance while handling complex automotive engineering queries.

2.1 Application Design

The MultiAgent Automotive system follows a microservices-inspired architecture with specialized agent-based components. Each agent operates independently while communicating through a centralized state management system.

Step-by-Step Process Description:

1. **Query Reception**: User submits natural language query through the interface
2. **Intent Analysis**: Supervisor agent parses query, identifies comparison parameters
3. **Research Orchestration**: Supervisor activates researcher agent with specific parameters
4. **Data Collection**: Researcher agent scrapes multiple automotive websites and APIs
5. **Data Validation**: Collected data is validated for accuracy and completeness
6. **State Update**: Validated data is stored in shared state memory

7. **Writer Activation**: Supervisor triggers writer agent with formatting requirements
8. **Content Generation**: Writer creates structured content in requested formats
9. **Quality Assurance**: Final output verified against requirements
10. **Delivery**: Formatted document delivered to user

2.1.1 Architectural Style

The Entertainment Planning Assistant Agent follows a **Modular Layered Architecture with Agent-Oriented Design**.

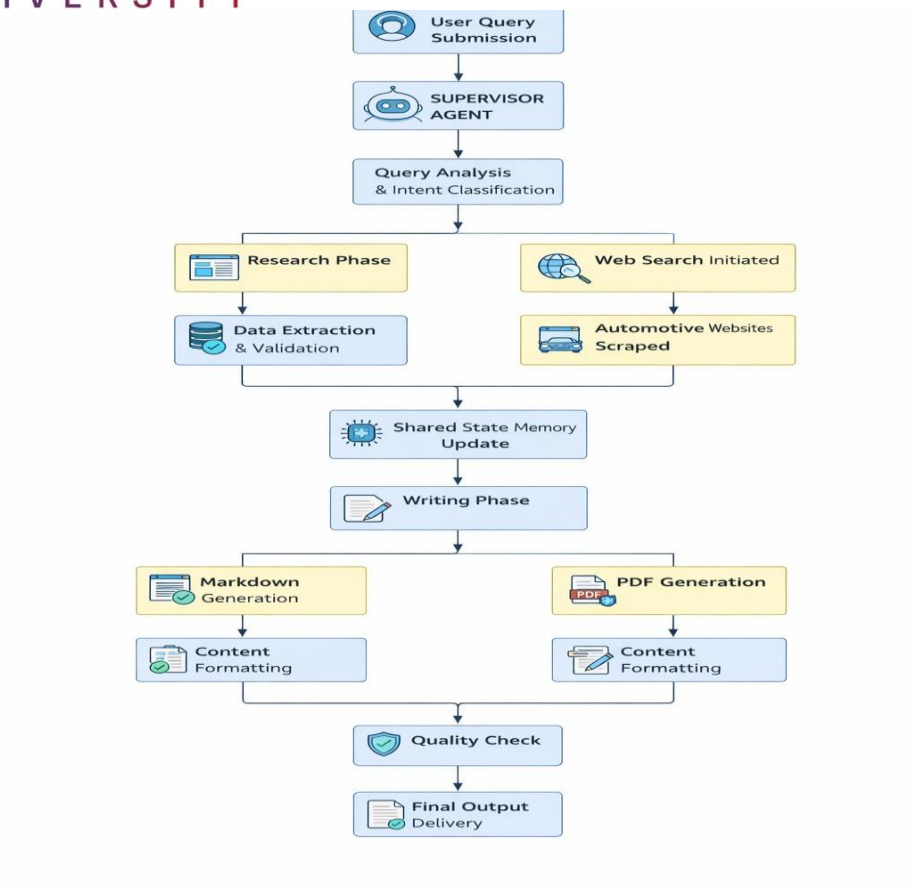
The architecture is divided into logical layers:

1. **Presentation Layer** (Input/Output handling)
2. **Agent Orchestration Layer**
3. **Agent Researcher Layer**
4. **Agent writer Layer**
5. **Tool Integration Layer**
6. **Data Management Layer**
7. **Output Layer**

This separation ensures:

- High maintainability
- Easy module replacement
- Clear separation of concerns
- Extensibility for future enhancements

2.1.2 High-Level Application Architecture



2.1.3 Core Modules

1. Planner Agent Orchestrator

- Controls reasoning loop
- Manages memory/state
- Calls tools dynamically
- Handles error recovery

Technology :-

- Python
- LangChain / CrewAI
- LLM API

2. Task Decomposition Module

- Converts high-level goal into structured subtasks
- Produces JSON-based task objects
- Assigns task metadata (priority, estimated time, dependencies)

3. Dependency Resolver

- Builds Directed Acyclic Graph (DAG)
- Detects cycles
- Applies topological sorting
- Ensures correct execution order

4. Tool Execution Layer

- Simulates external systems
- Validates data/resource availability
- Returns structured responses
- Implements retry mechanism

5. Schedule Optimizer

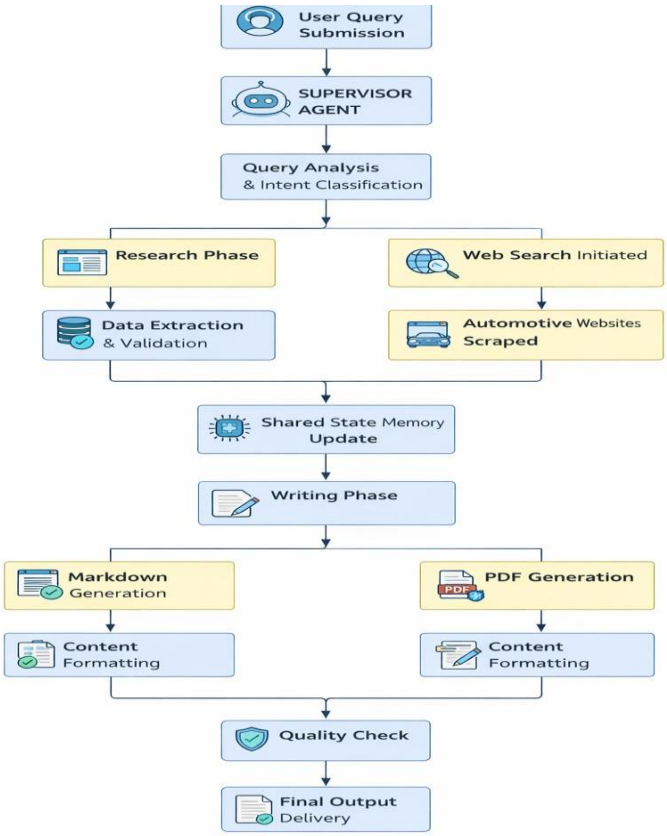
- Evaluates task sequence
- Optimizes parallel execution
- Generates time-based data
- Minimizes idle time

2.2 Process Flow

The system follows a sequential yet flexible process flow that adapts to different query types and user requirements.

2.2.1 High-Level Process Flow

The system follows a structured multi-step reasoning process:



2.2.2 Detailed Step-by-Step Process

1. **Query Reception**: User submits natural language query through the interface
2. **Intent Analysis**: Supervisor agent parses query, identifies comparison parameters
3. **Research Orchestration**: Supervisor activates researcher agent with specific parameters
4. **Data Collection**: Researcher agent scrapes multiple automotive websites and APIs
5. **Data Validation**: Collected data is validated for accuracy and completeness
6. **State Update**: Validated data is stored in shared state memory
7. **Writer Activation**: Supervisor triggers writer agent with formatting requirements
8. **Content Generation**: Writer creates structured content in requested formats
9. **Quality Assurance**: Final output verified against requirements
10. **Delivery**: Formatted document delivered to user

2.2.3 Exception Handling Flow

Scenario 1: Tool Failure

- Retry up to N attempts
- If failure persists → Trigger re-planning

Scenario 2: Dependency Error

- Detect cycle in DAG
- Re-generate task decomposition

Scenario 3: LLM Output Error

- Validate JSON schema
- Re-prompt with structured constraints

2.2.4 Multi-Step Reasoning Loop (Internal Process)

Internally, the agent runs:

While Goal_Not_Achieved:

Think (LLM reasoning)
Plan (Task decomposition)
Validate (Tool calls)
Observe (Results)
Refine (Adjust plan)

This enables:

- Adaptive planning
- Dynamic error recovery
- Intelligent task sequencing

2.3 Information Flow

This section explains how **data and information move across system components** within the Entertainment Planning Assistant Agent.

Unlike Process Flow (which explains steps), Information Flow focuses on:

- What data is generated
- How it is transformed
- Where it is stored
- How it moves between modules

2.3.1 High-Level Information Flow

```
User Input (Text)
        ↓
Supervisor Agent
        ↓
Task Assign
        ↓
Researcher Agent
        ↓
Shared Memory
        ↓
Writer Agent
        ↓
Supervisor Agent
        ↓
Output Data
```

2.5 Component-Level Architecture

Scalability Requirements:

Support for 1000+ concurrent user queries

Ability to add new data sources without system modification

Caching mechanisms for frequently requested comparisons

Performance Optimization:

Asynchronous processing for research tasks

Parallel scraping where legally permissible

Response time target: < 30 seconds for complete reports

Reliability Factors:

Graceful degradation when external sources unavailable

Data validation at multiple stages

Fallback mechanisms for failed requests

Extensibility:

Plugin architecture for new agent types

Configurable output format templates

Easy integration of new data sources

Compliance Requirements:

Respect robots.txt for web scraping

API rate limiting compliance

Data attribution and source citation

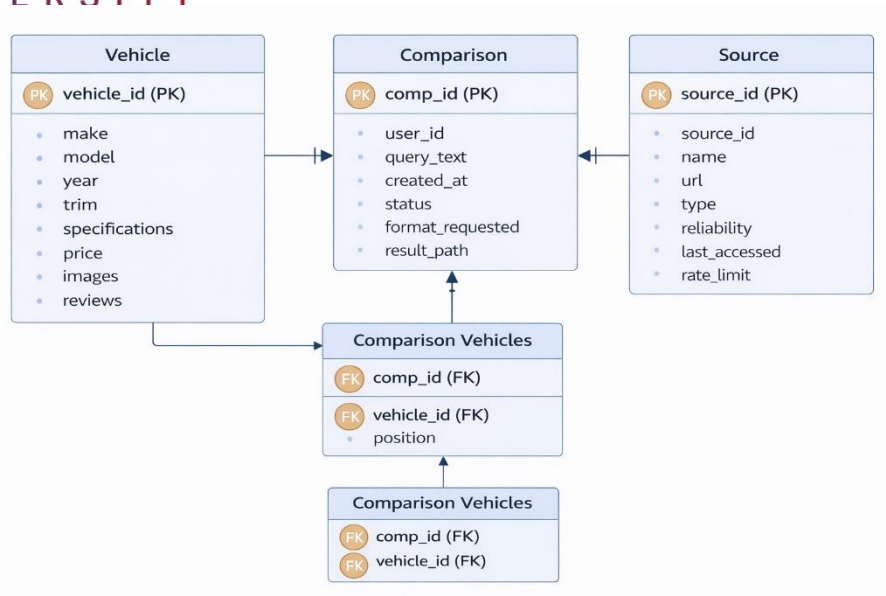
2.6 Key Design Considerations

This section outlines the major architectural and technical decisions taken while designing the **Multiagent Automotive Agent**.

These considerations ensure scalability, robustness, maintainability, and intelligent reasoning capability.

Data Model

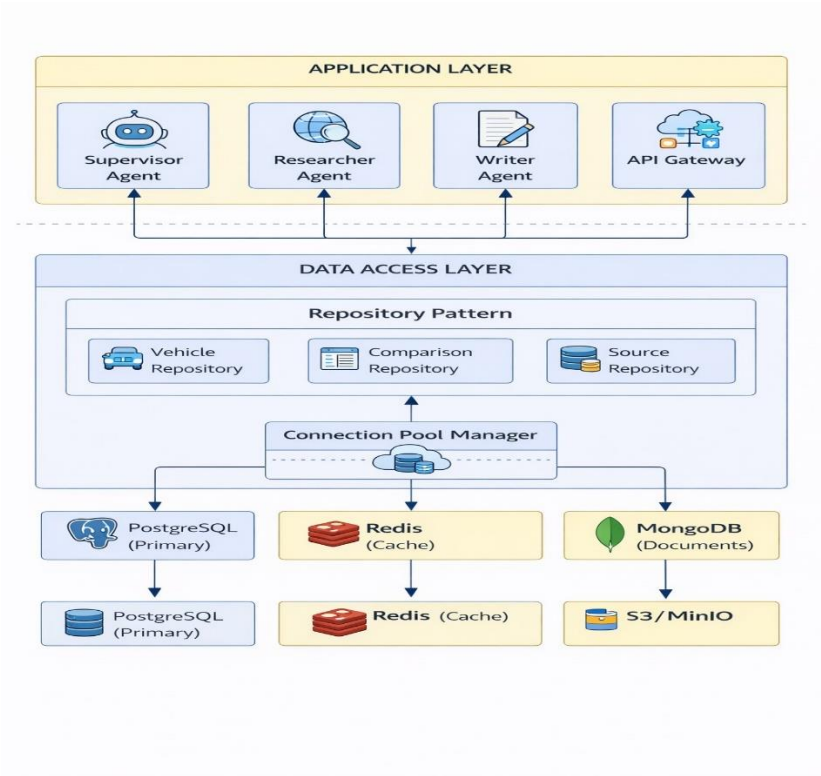
The data model for MultiAgent Automotive is designed to support complex vehicle comparisons while maintaining flexibility for future enhancements. It encompasses several interconnected entities that represent the automotive domain comprehensively.



3.2 Data Access Mechanism

The data access layer is designed to provide efficient, secure, and scalable access to the system's data while maintaining separation of concerns between agents.

Storage Architecture



Database	Purpose	Justification
PostgreSQL	Primary relational data	ACID compliance for transactional data, JSONB support for flexible specs robust querying
Redis	Caching & session management	In-memory speed for frequent access, TTL support, pub/sub for agent communication
MongoDB	Document storage	Schema flexibility for comparison reports, rich querying on nested documents
MinIO/S3	File storage	Scalable object storage for PDFs, images, and large attachments

2.6.1 Structured LLM Output

LLM responses are enforced in structured JSON format.

Reason:

- Prevent hallucinated unstructured text
- Enable machine validation
- Simplify dependency graph construction

Validation techniques:

- JSON schema validation
- Retry with corrective prompts

2.7 API Catalogue

This section describes the internal and external APIs used in the Entertainment Planning Assistant Agent system.

2.7.1 Goal Submission API

Endpoint: `/submit-goal`

Method: POST
Description: Accepts a high-level entertainment planning goal from the user.

Request:

```
{
  "goal": "Movie Reviews for latest releases"
}
```

Response:

```
{
  "task_id": "12345",
  "status": "Planning Initiated"
}
```

Purpose:

- Triggers Planner Agent
- Initiates reasoning loop

2.7.2 Task Status API

Endpoint: `/task-status/{task_id}`
Method: GET
Description: Retrieves the current execution state of the planning task.

Response:

```
{
  "task_id": "12345",
  "status": "Execution Scheduled",
  "progress": "70%"
}
```

2.7.3 Resource Validation API (Mock Tool)

Endpoint: `/validate-resource`
Method: POST
Description: Simulates validation of external entertainment resources (e.g., availability of movie database, show timings, streaming availability).

Request:

```
{
  "resource_type": "Movie Database",
  "resource_name": "IMDb"
}
```

Response:

```
{
  "available": true,
  "latency": "120ms"
}
```

2.7.4 LLM Interaction API

External Integration: LLM Provider (e.g., OpenAI API)

Purpose:

- Task decomposition
- Dependency resolution
- Schedule generation
- Optimization reasoning

Invocation Example (Conceptual):

```
llm.invoke(prompt)
```

3 Data Design

The data design focuses on storing planning goals, decomposed tasks, dependencies, execution schedules, and system metadata.

3.5 Data Model

The system follows a structured object-based model.

3.5.1 Entities

1. Goal

- goal_id (UUID)
- goal_description (Text)
- created_at (Timestamp)
- status (Enum: Initiated, Planning, Scheduled, Completed)

2. Task

- task_id (UUID)
- goal_id (Foreign Key)
- task_description (Text)

- priority (Integer)
- dependency_ids (List)
- status (Pending, Validated, Scheduled, Completed)

3. Resource

- resource_id
- resource_name
- resource_type
- availability_status

4. Execution Schedule

- schedule_id
- goal_id
- ordered_tasks (List)
- estimated_completion_time

3.6 Data Access Mechanism

The system uses:

- In-memory storage (for prototype implementation)
- Python dictionaries / JSON structures
- Optional SQLite for persistent storage

Access Layer Responsibilities:

- CRUD operations on goals and tasks
- Maintain dependency DAG
- Update task status during execution
- Retrieve schedules efficiently

LangChain/CrewAI memory modules may also maintain intermediate reasoning state.

3.7 Data Retention Policies

For Prototype Version:

- Data stored temporarily in memory
- Cleared after session termination

For Production Version:

- Retain goal and task history for 30–90 days
- Logs stored for auditing and debugging
- Sensitive data anonymized

Backup Strategy:

- Periodic snapshot of schedules
- JSON export of execution plans

3.8 Data Migration

Future migration plan:

- Move from in-memory storage to relational database (PostgreSQL)
- Schema versioning for data model evolution
- Use migration tools like Alembic (if SQL-based)
- Backward compatibility maintained via version-controlled schema

4 Interfaces

4.5 User Interface

- CLI-based interaction (initial version)
- Optional REST API endpoint
- Future: Web-based dashboard

User Inputs:

- High-level entertainment goal
- Constraints (optional time, platform preference)

Outputs:

- Task breakdown
- Dependency graph
- Execution timeline

4.6 External Interfaces

1. LLM API
2. Mock Resource Validation Tools
3. Optional future integrations:
    - Movie databases
    - Streaming platforms APIs

5 State and Session Management

The Planner Agent operates in a session-based context.

Session Data Includes:

- Current goal
- Intermediate reasoning steps
- Generated tasks
- Dependency graph
- Execution progress

State Management Strategy:

- Stored in memory during session

- LangChain conversation memory maintains reasoning history
- Unique task_id binds session state

Failure Handling:

- On failure, system retries reasoning loop
- Partial plan recovery supported

6 Caching

Caching improves performance and reduces LLM API cost.

Cacheable Components:

- Previously decomposed similar goals
- Resource validation results
- LLM responses for repeated prompts

Caching Strategy:

- In-memory dictionary cache
- Optional Redis (future enhancement)
- Cache invalidation after configurable TTL

Benefits:

- Reduced API latency
- Cost optimization
- Faster repeated planning tasks

7 Non-Functional Requirements

7.5 Security Aspects

1. API authentication (Token-based authentication)
2. Secure storage of LLM API keys
3. Rate limiting to prevent abuse
4. Input validation to prevent prompt injection
5. Logging with masked sensitive information

Threat Mitigation:

- Sanitize user inputs before sending to LLM
- Restrict tool invocation permissions
- Controlled execution environment for Python tools

7.6 Performance Aspects

Target Performance Goals:

- Goal decomposition < 3 seconds
- Resource validation < 500 ms (mock)
- Schedule generation < 5 seconds

Scalability:

- Horizontal scaling via microservice architecture
- Asynchronous task execution
- Load balancing for API layer

Optimization Techniques:

- Parallel resource validation
- Efficient DAG construction
- LLM prompt optimization

8 References

1. LangChain Documentation – https://python.langchain.com
2. CrewAI Documentation – https://docs.crewai.com
3. OpenAI API Documentation – https://platform.openai.com/docs
4. REST API Design Best Practices – Richardson Maturity Model